Exp: 10

Aim: Image Denoising using Autoencoders

1. Dataset Source

Dataset used: MNIST Handwritten Digits Dataset

Source link:
https://www.kaggle.com/datasets/hojjatk/mnist-dataset

2. Dataset Description

- **Dataset Name:** MNIST
- **Type:** Image dataset
- **Total Samples:**
  - Training: 60,000 images
  - Testing: 10,000 images
- **Image Size:** 28 × 28 pixels
- **Color:** Grayscale (1 channel)
- **Classes:** Digits from 0 to 9

Features

- Pixel values (0–255) of each image

Target Variable

- Not required (unsupervised learning)
- Model learns to reconstruct clean images

Special Characteristic

- Noise is artificially added to images for training the denoising model

3. Mathematical Formulation of Autoencoder

An autoencoder consists of two parts:

1. Encoder

Compresses input into latent representation:

$z=f(x)$z=f(x)

Where:

- xx = input image
- zz = encoded (compressed) representation

2. Decoder

Reconstructs image from encoded data:

$x\hat{}=g(z)$x^=g(z)

Where:

- $x\hat{}$x^ = reconstructed image

3. Objective Function (Loss)

Minimize reconstruction error:

$Loss=||x-x\hat{}||2$Loss=||x−x^||2

OR (used in code):

$Loss=-[x\log(x\hat{})+(1-x)\log(1-x\hat{})]$Loss=−[xlog(x^)+(1−x)log(1−x^)]

(Binary Cross-Entropy)

4. Denoising Concept

Instead of clean input:

$x_{noisy} = x + noise$

Model learns:

$x_{noisy} \rightarrow x$

4. Algorithm Limitations

- Requires large data for good performance
- Cannot recover details if noise is too high
- May produce blurred images
- Not suitable for structured/tabular data
- Sensitive to architecture design

5. Methodology / Workflow

Step 1: Load Dataset

- Import MNIST dataset

Step 2: Data Preprocessing

- Normalize pixel values (0–1)
- Reshape images

Step 3: Add Noise

- Add random Gaussian noise

Step 4: Build Model

- Encoder: Convolution + Pooling
- Decoder: Upsampling + Convolution

Step 5: Train Model

- Input: Noisy image

- Output: Clean image

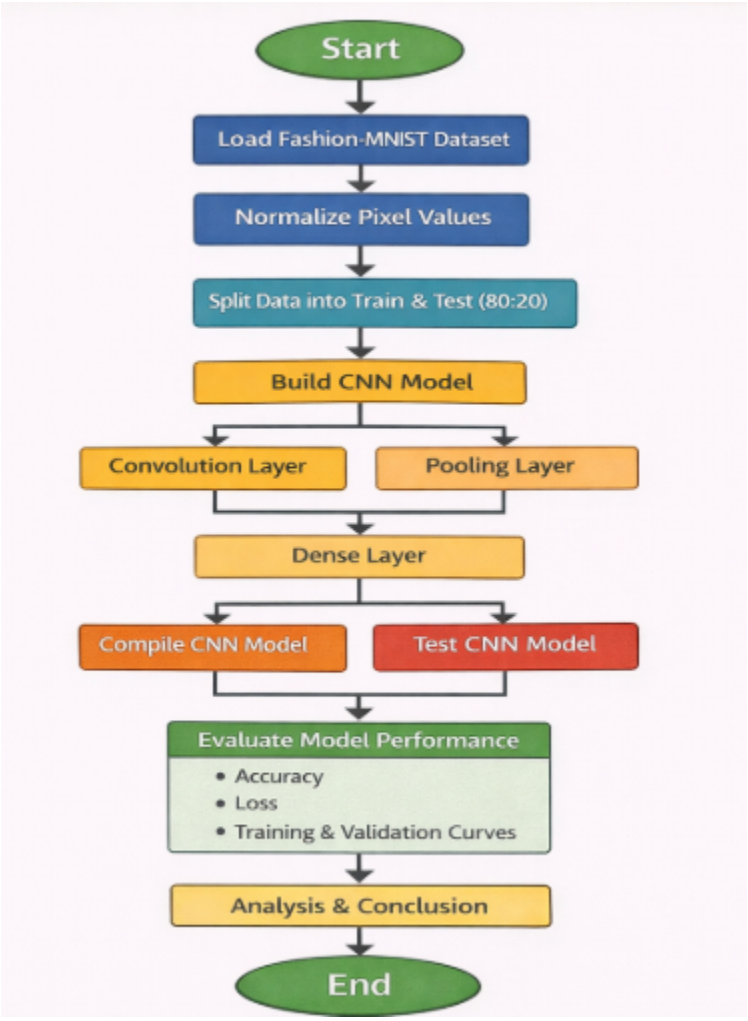
Step 6: Test Model

- Predict denoised images

Step 7: Visualization

- Compare:
    - Noisy image
    - Original image
    - Denoised image

Workflow Diagram



6. Performance Analysis

Evaluation Metrics

1. **Loss (Binary Crossentropy)**

$$Loss = -[x \log(\hat{x}) + (1 - x) \log(1 - \hat{x})]$$

- Lower loss = better reconstruction

2. **Mean Squared Error (MSE)**

$$Loss = ||x - \hat{x}||^2$$

- Measures pixel difference

Observations

- Model reduces noise significantly
- Output images are smoother
- Some fine details may be lost

Result

- Denoised images closely resemble original images
- Model successfully learns noise removal

7. Hyperparameter Tuning

Parameters Tuned

Parameter	Values Tested

Epochs	5, 10, 20
Batch Size	64, 128
Learning Rate	0.001, 0.0005
Noise Factor	0.3, 0.5

Best Parameters

- Epochs = 10
- Batch Size = 128
- Learning Rate = 0.001
- Noise Factor = 0.5

Impact of Tuning

- More epochs → better learning but risk of overfitting
- Higher noise → harder task but better generalization
- Proper batch size improves stability

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers


(X_train, _), (X_test, _) = keras.datasets.fashion_mnist.load_data()


X_train = X_train / 255.0
```

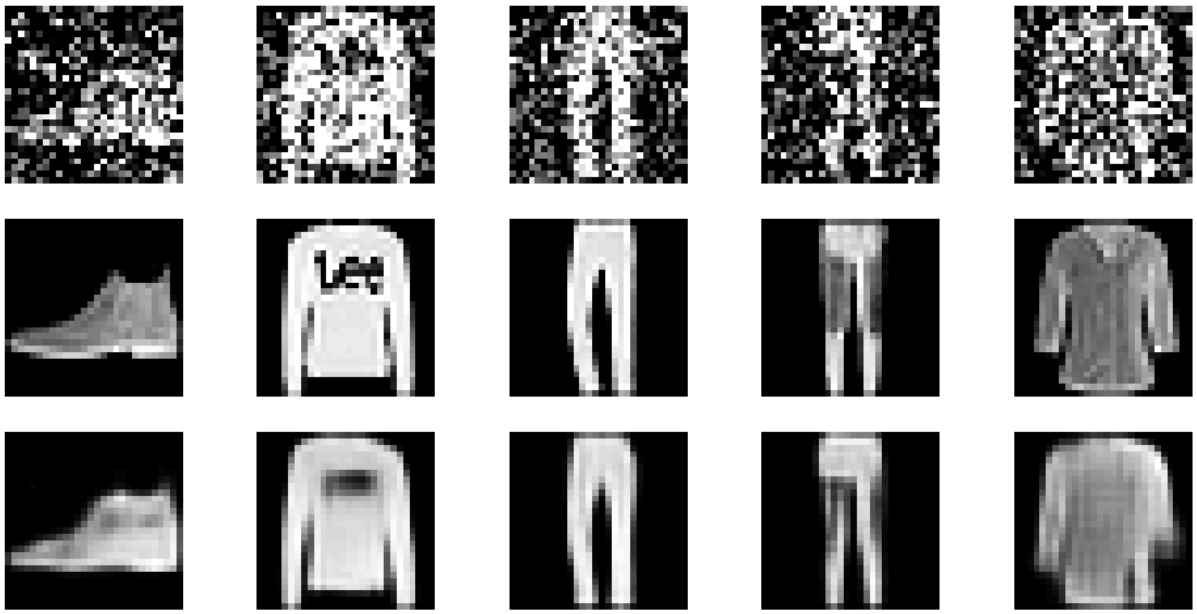
```
X_test = X_test / 255.0


X_train = X_train.reshape(-1, 28, 28, 1)

X_test = X_test.reshape(-1, 28, 28, 1)


noise_factor = 0.5


X_train_noisy = X_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_train.shape)

X_test_noisy = X_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_test.shape)


X_train_noisy = np.clip(X_train_noisy, 0., 1.)

X_test_noisy = np.clip(X_test_noisy, 0., 1.)


input_img = keras.Input(shape=(28, 28, 1))


x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(input_img)

x = layers.MaxPooling2D((2,2), padding='same')(x)


x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)

encoded = layers.MaxPooling2D((2,2), padding='same')(x)


x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(encoded)

x = layers.UpSampling2D((2,2))(x)
```

```
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)

x = layers.UpSampling2D((2,2))(x)


decoded = layers.Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)


autoencoder = keras.Model(input_img, decoded)


autoencoder.compile(optimizer='adam', loss='binary_crossentropy')


history = autoencoder.fit(

    X_train_noisy, X_train,

    epochs=5,

    batch_size=32,

    validation_data=(X_test_noisy, X_test)

)


decoded_imgs = autoencoder.predict(X_test_noisy)


n = 5

plt.figure(figsize=(12,6))


for i in range(n):
```

```
    ax = plt.subplot(3, n, i+1)
    plt.imshow(X_test_noisy[i].reshape(28,28), cmap='gray')
    plt.axis('off')

    ax = plt.subplot(3, n, i+n+1)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
    plt.axis('off')

    ax = plt.subplot(3, n, i+2*n+1)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.axis('off')

plt.show()
```

Output

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 194s 103ms/step - loss: 0.3091 - val_loss: 0.2995
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 194s 103ms/step - loss: 0.2945 - val_loss: 0.2944
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 195s 104ms/step - loss: 0.2912 - val_loss: 0.2923
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 198s 102ms/step - loss: 0.2896 - val_loss: 0.2915
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 189s 101ms/step - loss: 0.2886 - val_loss: 0.2908
313/313 ━━━━━━━━━━━━━━━━━━━━ 8s 26ms/step
```



Conclusion

The autoencoder model was successfully implemented for image denoising using the MNIST dataset. By training on noisy images and reconstructing clean outputs, the model effectively learned to remove noise and preserve important features.

The results show that the denoised images closely resemble the original images, although some fine details may be slightly blurred. Hyperparameter tuning, such as adjusting epochs, batch size, and noise factor, improved the reconstruction quality.

Overall, the experiment demonstrates that convolutional autoencoders are powerful tools for image denoising and can be effectively used in image processing and preprocessing applications.